

42 경산 교육과정 및 주요 프로젝트 요약

지원 직무: 임베디드 소프트웨어 개발 (Linux 시스템 프로그래밍)
수행 과제 정리 (GitHub) github.com/lus0312/systems-programming-portfolio

1. 교육과정 개요

교육 기관	42 경산 (이노베이션 아카데미)
학습 기간	2024.10 ~ 2026.10(예정) / 마지막 과제(ft_transcendence)만 남은 상태
수행 현황	개인 과제 35 개, 팀 과제 5 개 수행 / 미국 해외연수 참여 (2026.01 ~ 03)
사용 기술	C, C++98, Linux, GNU Make, Valgrind, Docker Compose, Git
학습 방식	• 자기주도 학습: 수업과 강사 없이 단계별 과제와 테스트를 진행합니다. 과제 명세(subject)를 읽고 요구사항을 해석해 범위를 정하고, 필요한 자료를 직접 조사하며 과제를 수행합니다. • 동료 평가: 과제별 최소 3 명 이상의 동료에게 코드를 설명하고 평가받으며, 동료의 코드를 리뷰하고 피드백을 주고받습니다. • 규범 준수: 42 만의 코드 스타일 규칙(Norm)을 지켜야 하고, 비정상 종료나 메모리 누수가 없도록 과제를 완료합니다. 과제별로 허용 함수가 제한되어 있어, 표준 함수를 쓸 수 없는 경우에는 직접 구현합니다. • 팀 프로젝트: 2 ~ 3 인 팀으로 역할을 나누어 개발합니다. (minishell 2 인, cub3D 2 인, ft_irc 3 인) • 다양한 주제 학습: C 라이브러리 · 알고리즘 · 프로세스와 스레드 · 네트워크 · 컨테이너 · 그래픽 · C++ 객체지향 등 여러 분야의 주제를 과제로 다루며 폭넓게 학습합니다.

2. 수행한 과제 (Rank 별)

Rank	과제	설명
00	libft	문자열 · 메모리 · 문자 판별 등 C 표준 라이브러리 함수와 연결 리스트 함수를 직접 구현하는 과제입니다. 이를 통해 C 언어의 기본 문법과 포인터, 메모리 관리의 기초를 익혔습니다.
01	ft_printf	가변 인자를 이용해 printf 의 변환(c s p d i u x X %)을 직접 구현하는 과제입니다.
01	get_next_line	fd 에서 데이터를 한 줄씩 읽어 반환하는 함수를 구현한 과제입니다. 정적 변수와 동적 메모리 관리, 파일 입출력 방식에 대해 학습했습니다.
01	Born2beRoot	가상 머신에 리눅스 서버를 설치하고 사용자, 권한, 보안 정책 및 서비스를 설정하는 과제입니다. 리눅스 시스템 관리, 권한, 네트워크 및 보안 설정의 기초를 학습했습니다.
02	push_swap	스택 두 개와 제한된 명령어만으로 두 개의 스택에 저장된 데이터를 최소 명령 수로 정렬하는 알고리즘 과제입니다. 정렬 알고리즘과 자료구조, 연산 횟수를 고려한 알고리즘 최적화에 대해 학습했습니다.
02	pipex	셸의 파이프 동작을 C 로 구현하여 여러 명령어의 표준 입출력을 연결하는 과제입니다. fork, pipe, dup2, execve 를 활용하며 프로세스와 파일 디스크립터의 동작 원리를 익혔습니다.
02	fract-ol	Mandelbrot 와 Julia 등 프랙탈을 화면에 시각화하고 확대·이동 기능을 구현하는 그래픽 과제입니다. 복소수 계산, 좌표 변환, 반복 알고리즘과 그래픽 이벤트 처리 방식을 학습했습니다.
03	Philosophers	식사하는 철학자 문제를 멀티스레딩으로 구현하며 데드락과 경쟁 상태를 방지하는 과제입니다. 스레드와 뮤텁스를 이용한 동시성 제어와 공유 자원 관리 방법을 학습했습니다.
03	minishell (2 인)	bash 의 일부 기능을 직접 구현하여 명령어 실행, 파이프, 리다이렉션, 환경 변수, 시그널 등을 처리하는 셸을 만드는 과제입니다. 프로세스 생성과 제어, 파일 디스크립터, 파이프 및 Unix 시스템 프로그래밍을 학습했습니다.
04	NetPractice	IP 주소 · 서브넷 마스크 · 라우팅을 직접 설정해 소규모 네트워크의 모든 장치가 서로 통신하게 만드는 10 단계의 문제를 15 분 안에 푸는 과제입니다. IP 주소, 서브네팅, 라우팅 등 TCP/IP 네트워크의 기본 원리를 익혔습니다.

04	cub3D (2 인)	레이캐스팅 기법을 이용해 2D 맵 데이터를 1 인칭 3D 화면처럼 표현하는 그래픽 과제입니다. 좌표 계산, 벡터와 시야각 개념, 충돌 처리, 텍스처 매핑과 실시간 렌더링 구조를 학습했습니다.
04	C++ Module 00 ~ 04	클래스, 생성자와 소멸자, 상속, 추상 클래스, 다형성 등을 단계적으로 구현하는 과제입니다. C++의 객체지향 프로그래밍 구조와 객체의 생명주기에 대해 학습했습니다.
05	C++ Module 05 ~ 09	예외 처리, C++ 형 변환, 템플릿, STL 컨테이너 · 반복자 · 알고리즘을 활용해 다양한 프로그램을 구현하는 과제입니다.
05	ft_irc (3 인)	poll() 기반의 논블로킹 I/O 를 사용하여 여러 클라이언트를 동시에 처리하는 IRC 서버를 구현하는 과제입니다. 소켓 통신, 비동기 네트워크 I/O, 프로토콜 설계와 다중 클라이언트 관리 방식을 학습했습니다.
05	Inception	Docker 와 Docker Compose 를 이용해 NGINX, WordPress, MariaDB 를 각각 독립된 컨테이너로 구성하고 연결하는 인프라 구축 과제입니다. 컨테이너, Docker 네트워크, 볼륨, TLS 및 서비스 간 통신 구조, Redis, FTP 를 학습했습니다.
06	ft_transcendence	여러 기술을 조합해 웹 서비스를 만드는 종합 과제입니다. 현재 진행 중입니다.

이름을 굵게 표시한 과제는 다음 장에서 더 자세히 설명하며, 코드와 README 는 GitHub 저장소에서 볼 수 있습니다.

3. 주요 프로젝트

ft_irc — 다중 클라이언트 IRC 서버 C++98 · Linux · 3 인 팀	
개요	IRC 프로토콜을 기반으로 여러 클라이언트가 채널과 개인 메시지를 통해 통신할 수 있는 서버를 구현했습니다. 하나의 poll() 이벤트 루프에서 모든 클라이언트의 접속 · 수신 · 송신 · 종료를 처리했으며, TCP 가 메시지 경계를 보장하지 않고 I/O 가 즉시 완료되지 않을 수 있다는 점을 고려해 서버를 설계했습니다.
담당	소켓 및 연결 관리, 사용자 인증 · 등록, IRC 명령 실행, 채널 관리, 메시지 전달, 연결 종료를 담당했습니다. 팀원은 수신 메시지의 명령어 및 인자 파싱을 담당했습니다.
문제와 해결	수신이 느린 클라이언트가 서버 전체를 막는 문제 메시지 생성과 실제 전송을 분리하고, 전송하지 못한 데이터는 클라이언트별 송신 버퍼에 유지하고 이후 해당 소켓에서 POLLOUT 이벤트가 발생했을 때 남은 데이터를 이어서 전송하도록 구성했습니다. EAGAIN 또는 EWOULDBLOCK 발생 시에도 즉시 반복 호출하지 않고 버퍼를 유지한 채 다음 POLLOUT 이벤트를 기다리도록 했습니다. 그 결과 특정 클라이언트의 수신이 지연되어도 다른 클라이언트의 통신은 계속 처리할 수 있었습니다. 자체 서버의 응답만으로는 구현이 맞는지 판단하기 어려운 문제 테스트 과정에서 실제 IRC 서버인 Libera.Chat 의 응답과 상태 변화를 비교 기준으로 사용했습니다. 단순히 "동작한다"는 것보다 실제 IRC 서버가 동일한 상황에서 어떤 Numeric Reply 와 상태 변화를 발생시키는지 확인하며 구현을 검증했습니다.
배운 점	이 과제를 통해 네트워크 프로그램에서는 정상적인 요청뿐 아니라 I/O 가 지연되거나 데이터가 예상과 다른 단위로 전달되는 상황까지 상태로 관리해야 한다는 점 을 배웠습니다. 또한 사용자별 인증 상태와 송수신 버퍼 같은 연결별 상태 와, 채널 참여자 · 운영자 · 모드처럼 여러 사용자가 함께 사용하는 공유 상태 를 분리하여 설계하는 경험을 했습니다.
minishell — bash 축소 구현 C · Linux · 2 인 팀	
개요	bash 의 일부 동작을 직접 구현한 셸 프로그램입니다. 명령어 실행뿐 아니라 파이프, 리다이렉션, 빌트인 명령, 시그널, heredoc, 종료 상태 등을 처리하며 Unix 의 프로세스와 파일 디스크립터 동작을 구현했습니다.
담당	실행부 : 프로세스 생성 · 회수, 파이프, 리다이렉션 fd 처리, 빌트인, 시그널, heredoc, 종료 상태
문제와 해결	같은 Ctrl-C 가 상황마다 다르게 동작해야 하는 문제 셸에서 Ctrl-C 는 항상 같은 의미를 가지지 않았습니다. 프롬프트 입력 중에는 현재 입력만 취소하고 새 프롬프트를 보여야 하고, 외부 명령 실행 중에는 실행 중인 자식 프로세스가 종료되어야 하며, heredoc 입력 중에는 heredoc 을 중단하고 현재 명령 실행 자체를 취소해야 했습니다. 시그널 핸들러 안에서 복잡한 자료구조를 직접 변경하지 않고 최소한의 상태만 기록하도록 설계했습니다. 부모 셸은 현재 상태와

	자식 종료 결과를 확인해 프롬프트 복구와 <code>\$?</code> 값을 결정하도록 했습니다. 자식 프로세스에서는 실행 전 시그널 동작을 기본값으로 복원하여 실제 <code>bash</code>와 유사하게 동작하도록 구성했습니다. • heredoc 입력 중 Ctrl-C 처리 입력을 받는 일을 별도 자식 프로세스로 분리했습니다. 자식은 시그널 기본 동작으로 즉시 종료되고 입력은 파이프로 전달하며, 부모는 중단 여부에 따라 명령을 취소하거나 파이프의 읽는 끝을 명령의 입력으로 연결되도록 처리했습니다.
배운 점	이 과제를 통해 <code>fork</code>, <code>execve</code>, <code>pipe</code>, <code>dup2</code>, <code>waitpid</code>, 시그널이 각각 독립적인 기능이 아니라 프로세스와 파일 디스크립터의 생명주기를 함께 관리해야 하는 하나의 실행 구조라는 점을 이해했습니다. 또한 같은 입력이라도 셸의 현재 상태에 따라 처리 방식이 달라지기 때문에, 복잡한 프로그램에서는 현재 상태와 자원의 소유 관계를 명확히 설계하는 것이 중요하다는 점을 배웠습니다.

Philosophers — 스레드와 동기화 C · POSIX 스레드 · 개인

개요	Dining Philosophers 문제를 기반으로 여러 철학자 스레드가 식사 · 수면 · 사고를 반복하는 시뮬레이션을 구현했습니다. 각 철학자는 식사를 위해 양쪽 포크를 필요로 하며, 제한 시간 안에 식사하지 못하면 사망합니다. 여러 스레드가 동시에 공유 자원과 상태에 접근하는 환경에서 경쟁 상태와 데드락을 방지하는 것이 핵심 과제였습니다.
문제와 해결	• 여러 철학자가 동시에 포크를 기다리며 멈추는 데드락 모든 철학자가 동일한 순서로 첫 번째 포크를 획득한 뒤 두 번째 포크를 기다리면 서로가 가진 포크를 기다리는 순환 대기 상태가 발생할 수 있습니다. 철학자에 따라 포크 획득 순서를 다르게 구성하여 순환 대기 조건을 제거했습니다. 이를 통해 단순히 공유 자원을 뮤텍스로 보호하는 것과 여러 뮤텍스를 어떤 순서로 획득할지 설계하는 것은 별개의 문제라는 점을 확인했습니다. • 초기화 실패 시 실행 중인 스레드가 사용하는 자원이 먼저 해제될 수 있는 문제 구현을 다시 검토하는 과정에서 스레드 생성 도중 실패하면 이미 생성되어 실행 중인 스레드가 사용하는 뮤텍스와 메모리가 먼저 해제될 수 있는 경로를 발견했습니다. 실패 시 먼저 종료 상태를 설정하고, 실제로 생성된 스레드만 <code>join()</code>한 뒤 자원을 해제하도록 종료 순서를 변경했습니다. 뮤텍스 역시 초기화에 성공한 개수를 추적하여 성공한 자원만 해제하도록 보유했습니다. 정상 실행뿐 아니라 부분적으로 초기화된 상태에서도 안전하게 종료될 수 있도록 자원의 생명주기를 관리했습니다.
배운 점	이 과제를 통해 멀티스레드 환경에서는 단순히 뮤텍스를 추가하는 것만으로 안전성이 보장되지 않으며, 공유 데이터의 접근 규칙, 락 획득 순서, 스레드 종료 순서와 자원의 생명주기를 함께 설계해야 한다는 점을 배웠습니다. 또한 정상 실행 경로뿐 아니라 초기화 실패나 종료 중인 스레드가 존재하는 상황까지 고려해야 안전한 동시성 프로그램을 만들 수 있다는 점을 경험했습니다.

pipex — 셸의 파이프 구현 C · Linux · 개인

개요	<code>./pipex infile "grep a1" "wc -w" outfile</code> 이 셸의 <code>< infile grep a1 wc -w > outfile</code> 과 같은 결과를 내도록 만드는 과제입니다. 셸이 대신 처리해 주던 프로세스 생성, 실행 파일 탐색, 표준 입출력 연결과 종료 상태 처리를 직접 구현하며 Unix 프로세스와 파일 디스크립터의 동작을 학습했습니다.
문제와 해결	• 사용하지 않는 파이프 fd 때문에 프로그램이 종료되지 않는 문제 <code>fork()</code>를 호출하면 파일 디스크립터도 자식 프로세스에 복제됩니다. 따라서 부모나 다른 자식이 파이프의 쓰기 끝을 하나라도 열어둔 상태라면 읽는 프로세스는 아직 데이터가 들어올 가능성이 있다고 판단해 EOF를 받지 못하고 계속 기다릴 수 있습니다. 부모와 각 자식이 자신에게 필요하지 않은 파이프 끝을 명확하게 닫도록 구성하여 문제를 해결했습니다. • 과제에 명확히 정의되지 않은 동작의 기준을 정하는 문제 입력 파일이 존재하지 않는 경우, 실행 파일을 찾을 수 없는 경우, 직접 경로가 전달되는 경우 등 여러 예외 상황에 대해 실제 셸의 동작을 기준으로 삼았습니다. 셸과 동일한 입력을 나란히 실행하며 종료 상태와 출력 결과를 비교하여 동작 범위를 구체화했습니다.
배운 점	이 과제를 통해 셸의 파이프라인이 특별한 통신 구조가 아니라 프로세스의 표준 입출력을 파이프의 파일 디스크립터로 교체해 연결한 구조라는 것을 이해했습니다. 또한 프로세스를 여러 개 사용하는 프로그램에서는 <code>fork()</code> 이후 어떤 프로세스가 어떤 파일 디스크립터를 소유하고 언제 닫을지를 명확히 설계해야 프로그램의 종료와 데이터 흐름을 예측할 수 있다는 점을 배웠습니다.